Appendix of "QUITE: A Query Rewrite System Beyond Rules with LLM Agents"

A SQLStorm Workload Details

To test generalization, we evaluate QUITE on the SQLStorm benchmark [6] instantiated on StackOverflow-Math [7], which features real-world data (up to Oct. 2024) and LLM-synthesized queries beyond the distributions of standard benchmarks [1, 4, 10].

Query Generation. SQLStorm generates SQL queries of varying complexity (P1–P7) for benchmarking. We randomly sampled 20 queries per tier from P1–P5, as P6–P7 yielded low generation success rates [6]. Each query is executed to filter out syntactic errors. LLM-generated queries use general dialects that the rule-based Calcite engines in [5, 8, 11] cannot parse or rewrite directly (e.g., the MySQL-style `INTERVAL 1 YEAR` instead of `INTERVAL '1' YEAR`, or PostgreSQL's `STRING_AGG` instead of `LISTAGG`). We therefore manually normalized dialect inconsistencies for compatibility across Calcite and PostgreSQL, discarded unresolvable cases, and retained 43 valid queries.

Rewrite Performance. As shown in Table 3 of the paper and Table A1 (Imp.: improved queries; Eq.: equivalence ratio), QUITE outperforms all baselines on the SQLStorm benchmark. With hint injection, QUITE reduces average execution time by 72.7%, 72.6%, 74.2%, and 53.5% compared to LR, LLM-R^2, R-Bot, and LLM Agent, respectively (Section 7.2 of the paper). Even without hint injection (Table A1), the reductions remain 72.2%, 72.1%, 73.8%, and 52.6%, and the reduction relative to the original queries is 73.1%. Rule-inventory-based rewriters [5, 8, 11] often fail to generate valid rewrites even after manual dialect normalization, because they rely on rewrite engines such as Apache Calcite [2] that enforce strict SQL dialect constraints. QUITE is not tied to a single parser dialect.

Table A1: Different Methods on SQLStorm Benchmark

Methods	SQLStorm (13.8 GB)				Metrics	
	Mean	Med.	75th	95th	Imp.	Eq.(%)
Original	47.77	9.25	20.55	300.00	-	-
LearnedRewrite	46.25	10.90	19.33	300.00	3	72.1
LLM-R^2 (DS-R1)	46.16	11.31	18.62	300.00	3	83.7
R-Bot (GPT-4o)	49.03	11.93	20.61	300.00	3	86.0
LLM Agent (DS-R1)	27.14	6.38	9.40	272.55	23	79.1
QUITE (w/o hints)	**12.86**	**5.16**	**9.05**	**16.11**	**28**	**90.7**

B Rewrite Analysis Details

High-Impact Rewrites. We count the number of high-impact rewrites (speedup $> 10\times$) across five categories (a query may use several categories): CTE (26/34), Predicate (19/34), Join (10/34), Others (10/34), and Constant (5/34). We further classify each query by its applied optimization strategies and compute the average speedup to mitigate bias from mixed techniques. "CTE for Nested Subquery Rewrite" achieves an average speedup of 74.2×, "Join or Filter Pushdown into CTE" yields 66.8×, and "Join Order Optimization" provides 44.2×. Decorrelating nested subqueries into CTEs yields the largest average speedups, likely because it removes repeated evaluation of the same subquery [3, 9].

We also examine which query structures QUITE optimizes best. Queries with heavy aggregation, nesting, or late filtering show 10×–223× speedups after rewriting. In DSB and SQLStorm, QUITE decomposes large CTEs and removes redundant calculations, transformations that the rule sets of the evaluated rule-based systems [5, 8, 11] do not include. For Calcite queries, QUITE removes redundant subqueries and simplifies GROUP BY/UNION, and the resulting rewrites remain execution-consistent. Some of these strategies also exist as rewrite rules, but pattern matching often fails to match them to real-world queries. QUITE applies them flexibly with LLMs across diverse query patterns.

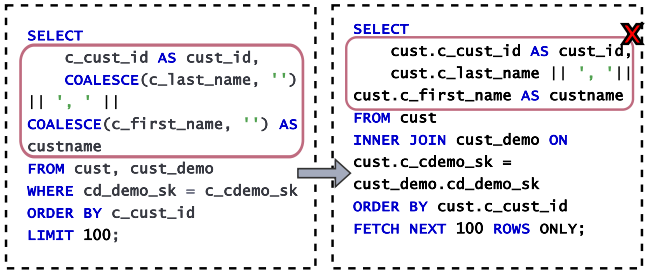
Q&A Units Analysis. We analyzed the retrieval frequencies of our 241 Q&A units on the DSB benchmark. Units from the "CTE" and "Join" categories were retrieved most frequently, at 197 and 191 times, respectively. One unit covered decorrelation (rewriting correlated subqueries into CTEs). It was retrieved 31 times, and 6 of the queries that retrieved it were improved. Our agent does not rank retrieved units by potential impact, because the benefit of a strategy depends on context and the same strategy can degrade another query.

Query Hint Analysis. On DSB, we analyze the frequency and impact of the most beneficial hints. A reduction in execution time of at least 10% is considered an improvement, while changes within ±10% are regarded as comparable performance. For join strategies, the `NoNestLoop` hint improved performance in 10 out of 25 cases and achieved comparable performance in 13 cases. The `HashJoin` hint improved 9 out of 19 cases and achieved comparable performance in another 9. For common table expressions (CTEs), the `NO_MATERIALIZE` hint was selectively applied in 6 cases, improving 4 and maintaining performance in 2. In cardinality correction, the `Rows` hint was the most frequently applied (27 cases) and accounted for the largest share of improvements (18 improved, 6 stable), consistent with cardinality misestimation being a common source of suboptimal plans.

C Rewrite Examples

Rewrites beyond Rule Inventories. On a Calcite query, QUITE composes predicate propagation, aggregation simplification, and projection pruning. The rewrite reduces runtime from 34.91s to 0.74s and SQL length from 219 to 133 characters. All three R-Bot variants instead produce a 369-character rewrite that fails to execute on PostgreSQL, because it references a select-list alias inside a WHERE EXISTS clause, so the original query is kept. On a SQLStorm query, QUITE removes output-irrelevant work across four CTEs. It reduces joins from 5 to 2, GROUP BY clauses from 3 to 0, window functions from 2 to 0, and runtime from the 300s timeout to 2.07s, while all R-Bot variants return the original query. No evaluated rule-based system produced either rewrite.

D Correctness Details



(a) Original Query Simplified from DSB Benchmark

(b) Rewritten Query using 'FILTER_INTO_JOIN' rule

Figure A1: A Nonequivalent Rewrite Produced by a Calcite Rule on PostgreSQL

A Nonequivalent Rule-Based Rewrite. Figure A1 shows an example from DSB. After Calcite's `FILTER_INTO_JOIN` rule is applied, the unparsed SQL drops the `COALESCE` wrappers around the name columns. On PostgreSQL, concatenating a NULL value yields NULL, so customers with a missing first or last name obtain a NULL `custname` instead of a partial name.

SQLSolver Verdicts. Section 7.5 of the paper reports the SQLSolver verdicts per benchmark. The nine NEQ outputs (one on Calcite and eight on SQLStorm) return identical results on the evaluated instance, so the execution-consistency gate cannot catch them and they count as Validated. The 11 outputs rejected by the execution check with hint injection are distributed as 1, 5, 1, and 4 on TPC-H, DSB, Calcite, and SQLStorm.

E Sensitivity Study

We examine the sensitivity of QUITE to the hint search space and to the choice of LLMs. We first test different model combinations. We fix DeepSeek-R1 as the reasoning model and vary the other models among Claude-3.7-Sonnet, DeepSeek-V3, and GPT-4o, and then we replace the reasoning model with DeepSeek-V3 and Claude-3.7-Sonnet. As shown in Table 4 of the paper, DeepSeek-R1 for reasoning combined with Claude-3.7-Sonnet for the other agents gives the lowest latency of all combinations (5.98s mean vs. 12.76–31.38s).

For the hint search space, as shown in Table 4 of the paper, we test the effect of adding the scan method to our hint base and the effect of using all hints (excluding GUC). The former yields a mean latency nearly identical to that of the unhinted rewrite (6.04s vs. 5.98s) and higher than that of our hint base (5.75s), while the latter increases mean latency from 5.75s to 7.06s, even above the unhinted rewrite (5.98s). This suggests that the size of our current hint search space is appropriate.

References

[1] 2021. Apache Calcite. https://github.com/georgia-tech-db/spes/blob/main/testData/calcite_tests.json Accessed: 2025-1-22.

[2] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J Mior, and Daniel Lemire. 2018. Apache calcite: A foundational framework for optimized query processing over heterogeneous data sources. In *Proceedings of the 2018 International Conference on Management of Data*. 221–230.

[3] Nicolas Bruno, César Galindo-Legaria, and Milind Joshi. 2025. Query Decorrelation in the Fabric Data Warehouse. In *Companion of the 2025 International Conference on Management of Data*. 297–309.

[4] Bailu Ding, Surajit Chaudhuri, Johannes Gehrke, and Vivek Narasayya. 2021. DSB: A decision support benchmark for workload-driven and traditional database systems. *Proceedings of the VLDB Endowment* 14, 13 (2021), 3376–3388.

[5] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2024. LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. *arXiv preprint arXiv:2404.12872* (2024).

[6] Tobias Schmidt, Viktor Leis, Peter Boncz, and Thomas Neumann. 2025. SQLStorm: Taking Database Benchmarking into the LLM Era. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4144–4157.

[7] StackExchange 2024. Stack Exchange Inc. https://math.stackexchange.com/ Accessed: 2025-10-07.

[8] Zhaoyan Sun, Xuanhe Zhou, and Guoliang Li. 2025. R-Bot: An LLM-based Query Rewrite System. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5031 – 5044.

[9] TiDB. 2025. Decorrelation of Correlated Subquery . https://docs.pingcap.com/tidb/stable/correlated-subquery-optimization/ Accessed: 2025-10-19.

[10] TPC 2024. TPC-H Toolkit. https://www.tpc.org/tpc_documents_current_versions/current_specifications5.asp Accessed: 2024-11-12.

[11] Xuanhe Zhou, Guoliang Li, Chengliang Chai, and Jianhua Feng. 2021. A learned query rewrite system using monte carlo tree search. *Proceedings of the VLDB Endowment* 15, 1 (2021), 46–58.